近似缓存用于高效服务扩散模型

Shubham Agarwal, Adobe Research, Subrata Mitra *, Adobe Research, Sarthak Chakraborty†, UIUC, Srikrishna Karanam, Adobe Research, Koyel Mukherjee, Adobe Research, Shiv Kumar Saini, Adobe Research

摘要： 利用扩散模型进行的文本到图像生成 因其能够根据文本提示生成高质量图像而广受欢迎。然而，扩散模型需要经过大量的迭代去噪步骤，并且资源消耗巨大，需要昂贵的 GPU，导致显著的延迟。在本文中，我们提出了一种新的 近似缓存技术，可以通过重复利用先前图像生成过程中创建的中间噪声状态来减少这些迭代去噪步骤。基于此想法，我们展示了一个端到端的文本到图像生成系统——NIRVANA，该系统使用近似缓存技术，并配有新颖的缓存管理策略，实现了 21% 的 GPU 计算节省、19.8% 的端到端延迟减少，以及在两个实际生产任务上节省了 19% 的成本。此外，我们还从缓存的角度，对生产环境中的实际文本到图像生成提示进行了广泛的特征分析，包括流行度和中间状态的重用。

1 介绍

文本到图像生成技术在过去几年中有了显著的进步 [31, 75]，如今已成为各大公司提供的一项广受欢迎的功能 [8, 14]，并整合到多个新的创意工作流程中 [10]。此类文本到图像模型的流行度巨大。Adobe 最近报告称 [10]，其 Firefly [9] 文本到图像服务已生成超过 20 亿张图像。类似的流行度也在 OpenAI 的 DALL-E 2 [12] 中得到了体现。图 2a 展示了随着时间推移，提交到基于稳定扩散（Stable Diffusion）文本到图像模型的门户中的提示数量的惊人增长，如 DiffusionDB 数据集 [69] 所捕获的。

尽管研究人员已经使用 VAEs [45]、GANs [37] 和其他技术 [34] 设计文本到图像生成模型已有一段时间，但当今最流行的文本到图像生产系统 [9, 17] 基于 扩散模型 [33, 42]。扩散模型被广泛采用，归因于它们能够生成高质量图像，并且能更准确地根据输入提示进行图像生成的条件设定。

扩散模型的问题：使用扩散模型进行图像生成是一个计算资源密集的任务，并且存在相对较长的延迟。在扩散模型的推理阶段（即从文本生成图像时），基本上是通过迭代去噪高斯噪声，以输入提示作为条件，通过 马尔可夫过程 [36] 生成输出图像。传统的扩散模型使用多达 1000 次此类迭代步骤 [33]。一些后续优化 [53, 66] 将这一过程缩短到大约 50-100 次迭代 [2]。

然而，即便是经过 50 步的扩散，使用扩散模型生成图像仍然是一个高资源消耗且缓慢的过程，

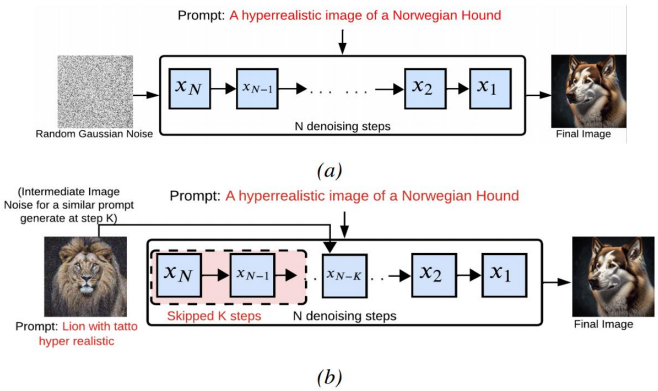
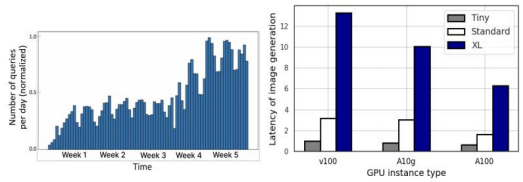


图 1：(a) 展示了基础扩散模型的工作原理。(b) 展示了具有近似缓存的扩散模型工作原理，在该模型中，前 K 个去噪步骤被跳过，因为从缓存中检索并重复使用了属于不同提示的中间噪声。

这不利于用户的互动体验，并且在昂贵的 GPU 上造成巨大的计算成本。

文本到图像生成的最新进展：在文本到图像生成中，给定一个描述图像某些期望特征的文本提示，一个深度神经网络模型会根据提示中提供的描

述生成图像。



(a) (b)

图 2：（a）展示了 DiffusionDB 中的工作负载在 5 周内的标准化增长。（b）在 AWS 上，展示了 Hugging Face [44] 仓库中的 Tiny、Standard 和 XL 稳定扩散模型在三种不同 GPU 架构下的延迟。较大的 XL 模型生成的图像质量远优于较小的两个模型，但伴随着显著较大的延迟开销，这也意味着更高的成本。

图 2b 展示了在 Amazon Web Services (AWS) 上可用的不同 GPU 架构（即 NVIDIA 的 A100、A10g 和 V100）在推理期间生成图像时，针对不同大小的扩散模型 (如：Stable Diffusion XL、Stable Diffusion 2.1、Hugging Face 仓库中的 Tiny Stable Diffusion [44]) 的延迟（以秒为单位），这些模型的典型迭代次数为 50 步。可以观察到，尽管较小的模型 (如 Tiny 和 Standard) 相比较大的模型 (如 XL）能显著降低推理延迟，但通常会伴随生成图像质量的显著下降 [58]。因此，为了提高用户体验而仅依靠缩小模型来减少延迟，可能会失去生成高质量图像的初衷。

延迟还可以通过使用更强大的 GPU 来减少，如图 2b 所示，但使用诸如 NVIDIA A100 这样更强大 GPU 的云实例价格显著较高。例如，在美国东部区域 [1]，V100 的价格大约为每小时 3.06 美元，A10g 为每小时 8.144 美元，而 A100 为每小时 32.77 美元，这使得 A100 实例比 A10g 贵 4 倍以上，且比 V100 贵超过 10 倍。因此，虽然可以通过更强大的 GPU 来减少推理延迟，但每张图像的推理成本也会大幅增加。

在本文中，我们介绍了一种高效的文本到图像生成系统，称为 NIRVANA，它通过一种称为 近似缓存 的新技术，智能地重用之前提示生成图像时创建的中间状态，显著减少计算成本和延迟。图 1 说明了 NIRVANA 的核心思想。对于图 1a 中红色显示的输入提示，展示了标准扩散模型如何通过多次迭代去噪步骤工作。在此过程中，首先从状态 xN 的高斯噪声开始，然后以输入提示为条件执行 N 次去噪步骤，最终在状态 x0 生成一致的图像。图 1b 显示了在 近似缓存 中，前 K 个步骤被跳过，直接使用从其他提示中检索到的适当的中间噪声。K 的值取决于新的输入提示与从中检索到噪声的提示之间的相似性。因此，计算量和延迟节省的程度可能会有所不同，这取决于缓存中是否有类似提示可用，而缓存中存储的是先前生成图像时产生的中间噪声或状态。近似缓存这个术语强调了在该系统中，我们并不直接重用从缓存中检索到的对象，而是检索中间状态，并进一步根据新的提示对其进行调整以生成图像。因此，NIRVANA 与基于检索的系统（如 GPT-CACHE [7]、PINECONE [4]）非常不同，后者是基于输入提示直接从缓存中检索图像。

在 NIRVANA 中用于选择特定输入提示的 K 值的启发式设计，实际上控制了命中率与计算节省之间的权衡。这里的命中率是指传入的提示在缓存中找到足够相似匹配的可能性，而 K 表示在使用缓存中检索到的噪声时，在扩散模型的初始阶段我们跳过的步骤数。如果我们打算在一开始跳过非常少的步骤（即 K 值较低），那么命中率可以显著提高。原因是，跳过较少的采样步骤，即使来自非常不相似提示的噪声仍然可以被有效调整以生成一致的图像。而在 K 较高的情况下，根据新提示修改噪声潜在空间的可能性会受到限制，但当能够实现时，可以带来巨大的计算节省。通过为每个传入的提示计算合适的 K 值，NIRVANA 的设计可以在保持高命中率的同时优化计算节省。

图 3 展示了使用 DiffusionDB [69] 中的提示进行的一些真实示例，说明了 NIRVANA 如何能够将来自看似不同的先前提示的噪声转化为一致且高质量的图像，同时显著减少延迟和计算量。

此外，在 NIRVANA 中，我们设计了一种新的近似缓存管理策略，称为 最小计算收益且常用（LCBFU），它以这样的方式管理存储的噪声，以确保在给定的缓存存储大小下，最大化性能和存储效率。我们优化了噪声的存储空间，以提供给

NIRVANA 最佳的计算效率。总体而言，NIRVANA 生成的图像质量与标准扩散模型(即不使用近似缓存) 非常接近，同时在 GPU 计算上节省了 21% 的开销，延迟减少了 19.8%，并在图像生成的花费上实现了 19% 的平均节省。我们对 60 名用户和 1000 张图像的研究表明，79% 的用户喜欢 NIRVANA 生成的图像，这远高于最佳基线 GPT-CACHE [7] 的 31% 的喜欢率，并且与昂贵且慢速的标准模型生成的图像质量非常接近，后者的用户喜欢率为 86%。

我们总结了本文的贡献如下：

1. 我们引入了近似缓存的创新理念，为文本到图像生成的扩散模型生产流程提供了显著的计算节省。
2. 我们提出了一种有效的缓存管理机制，称为 LCBFU，可以优化计算状态的重用和计算节省。
3. 我们展示了 NIRVANA 的端到端设计细节和理论依据，这是我们在云上优化的文本到图像部署系统。
4. 我们从可重用性和缓存的角度，对文本到图像模型的真实生产提示进行了特征分析。
5. 我们对两个真实且大型的文本到图像模型的生产提示进行了广泛评估，同时进行了人类评估和多项敏感性研究。

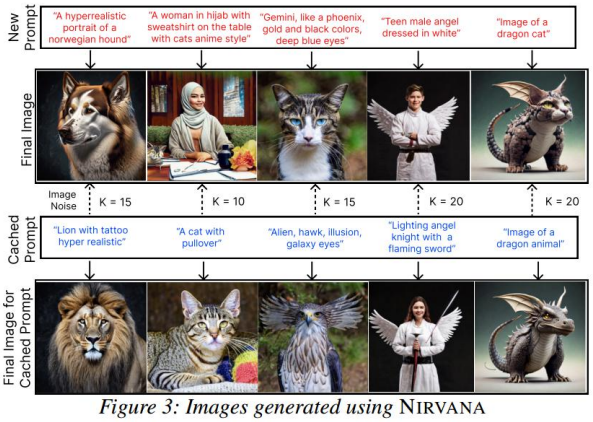


Figure 3: Images generated using NIRVANA

图 3：使用 NIRVANA 生成的图像

2 背景

2.1 扩散模型

扩散模型通过逐步去噪随机高斯噪声来生成一个基于文本条件的图像。训练过程包含一个前向扩散过程，该过程通过在马尔可夫过程中反复添加噪声来模糊图像，直到其饱和为高斯噪声。在反向扩散过程中，通过反复去除噪声来恢复原始图像。每一个这样的去噪步骤称为“采样”，因为模型通过去除噪声生成一个样本，而用于采样的方法称为采样器。

在前向扩散过程中，高斯噪声逐步添加到初始图像 x_0 中，经过 T 步骤后得到 x_T。根据马尔可夫链的假设，可以表达为：

$$q(x_{1:T}|x_0) := \prod_{t=0}^{T} q(x_t|x_{t-1}) \tag{1}$$

$$q(x_t|x_{t-1}) := \mathcal{N}(x_t|\sqrt{1-\beta_t}x_{t-1}, \beta_t I) \tag{2}$$

其中，$q(x_t|x_t-1)$是后验概率，$\beta_1,...,\beta_T$ 是噪声计划（可以是学习得到的或固定的），用于调节每个扩散步骤中的噪声水平。同样，反向扩散过程可以表示为：

$$p_\theta(x_{0:T}) := p(x_T)\prod_{t=0}^{T} p_\theta(x_{t-1}|x_t) \tag{3}$$

$$p_\theta(x_{t-1}|x_t) := \mathcal{N}(x_{t-1}|\bar{\mu}_t, \bar{\beta}_t I) \tag{4}$$

其中，$p\theta(.)$表示在给定 x_T 的情况下观察到 $xt-1$ 的概率。这里，$p(x_T)=N(x_T|0,I)$。这里的 μt 和 $\beta t\beta$是通过学习得到的。目标是学习 $p(\theta)$，以在反向/逆向扩散过程中最大化训练数据的似然性。最近的一些优化方法通过在预定的时间步上跳过某些中间状态来近似反向扩散过程 [47, 77]，从而将推理步骤从 $T\approx1000$ 减少到 $N\approx50$。这通过学习一个采样器实现，该采样器预测在每个扩散步骤后还剩下多少噪声，经过 T/N 步。即使有了这些优化，对于大模型的图像生成，仍然需要约 10 秒（在 A10g 上）或 6 秒（在 A100 GPUs 上），如图 2b 所示。

扩散模型的骨干基于 U-Net 架构 [61]。当给定文本提示时，图像生成过程通过模型内的交叉注意力进行条件化 [60]。因此，该模型构建了一个文本到图像的框架，能够基于文本描述生成视觉上

连贯且语境相关的图像。在实践中，每个 x_i 可以是实际图像，也可以是通过图像编码器计算的图像的潜在表示 [60]。通常，后一种方法称为潜在扩散模型（LDM），由于其能够捕捉图像的隐藏特征，因此更受欢迎。我们在工作中也使用了同样的方法。为了从逆向扩散结束时的潜在空间 x_0 生成最终图像，x_0 会通过图像编码器的逆过程进行解码。

2.2 图像生成动力学

根据K的不同，从缓存中检索的噪声为新提示进行再调整所需的程度会有所变化。我们观察到，图像的各种概念或特征，如布局、颜色、形状或物体、风格等，在某个 K值之后不容易进行修改。与最近的研究 [76] 类似，我们观察到，对于我们的数据集，LDM 模型 [60] 会在最初大约 20% 的扩散步骤内首先决定物体的布局。接下来，整个图像的色彩映射会在大约 40% 的步骤内确定，随后

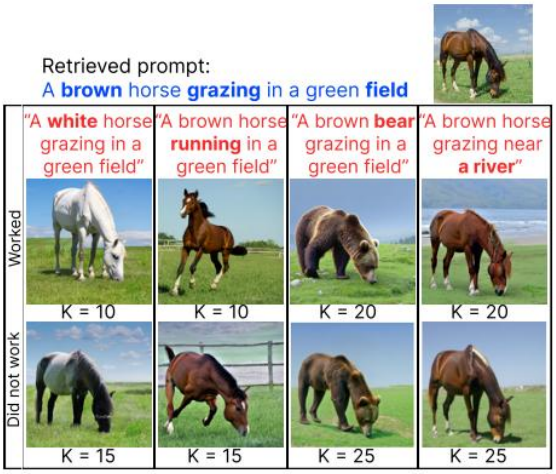
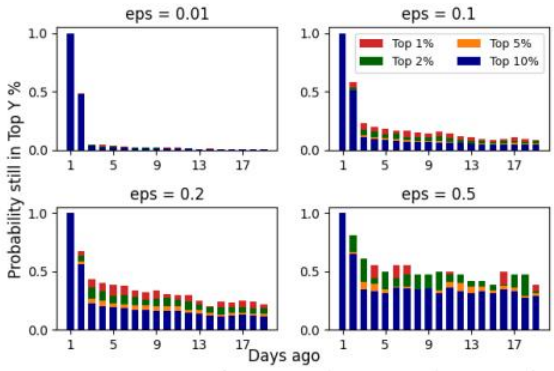


图 4：展示了棕色马的噪声成功在 K=10 时变为白马，而在 K=15 时未能实现相同效果。然而，噪声在 K=20 时竟然变成了一只熊。

是物体的形状和大小，最后是图像的艺术风格。总体来说，我们观察到大约在 50% 的步骤之后，图像的概念便会被冻结，此时对中间图像的任何再调整都不会在最终生成的图像中有所反映。例如，试图对一匹正在绿色草地上吃草的棕色马的图像进行再调整时（如图 4 所示），在K=10 时仍可以修改颜色，但在K=15 后就无法再进行修改了。同样，动作、物体和背景只能在某些步骤内修改，而无法在更晚的步骤中进行调整。

图 5：对于提示之间不同的相似度阈值，前 Y%流行提示集群



在几天内的流行度（eps 值）

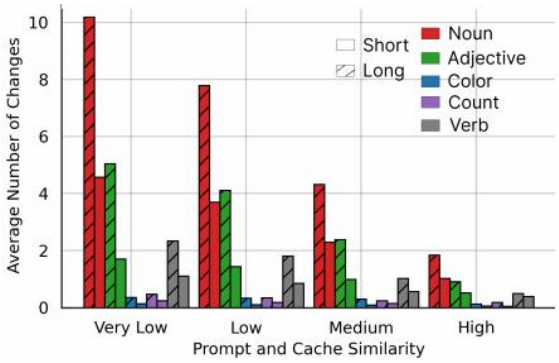


图 6：在短提示和长提示中，名词、形容词、颜色、计数和动词在相似性上的平均变化次数

3 理解用户提示信息

我们首先展示了对 DiffusionDB 数据集中实际用户提示的特征分析，该数据集包含了从 2022 年 8 月 8 日到 2022 年 8 月 22 日期间的 200 万个提示数据。每条记录都包含用户 ID、提示文本、时间戳以及生成的图像。

持续时间较长的前 Y%热门提示集群：我们使用 DBSCAN 算法（密度聚类方法）[64]对每一天的提示查询进行聚类，聚类依据的是提示的 CLIP 嵌入表示[59]，并使用了多个 eps 值（eps 控制两条数据点被视为邻居的最大距离）。我们找出了前 Y%（Y = 1、2、5 和 10）的热门提示集群，并提取了第 1 天中最热门的提示。接着，我们追踪这些热门提示集群在接下来几天（第 2 天到第 18

天）是否依然处于前 Y%之中，并将这一比例作为热门提示集群的持续概率报告在图 5 中。我们在 DBSCAN 算法中使用了不同的 eps 值（eps = 0.01, 0.1, 0.2, 0.5）重复这一分析。

结论：从图 5 中可以看出，当集群较为紧凑时（eps = 0.01），大多数处于前 Y%的热门提示集群在第 2 天甚至不会继续保持在前 Y%之内。尤其是在两天之后，集群的流行度急剧下降。然而，当我们增加 eps 值，使集群更加松散时，更多的提示被包含在同一个集群内，因此流行提示的衰减速率变慢。实验证明，随着 eps 的增加，更多且不相似的提示被聚合到一起，因此前 Y%集群的持续时间也更长。即便提示间相似性较低，近似缓存的思路依然可行，因为它能够对提取出的噪声进行重新调整。因此，在 eps = 0.2 和 0.5 下，较高的热门集群持续性显示出近似缓存的潜在优势。

最相似的短提示和长提示对：我们根据提示文本的字数百分比（大约 15 个词）将提示查询划分为短提示和长提示两类。对于每一类，我们通过它们的 CLIP 嵌入的余弦相似度形成最相似的提示对，并根据相似度得分将这些提示对分为 4 类：非常低（小于 0.65）、低（0.65 到 0.8）、中等（0.8 到 0.9）、高（大于 0.9）。然后，我们分析每类提示对中不同属性（如名词、形容词、动词、颜色、数量）发生了哪些变化，并展示了这些属性的平均变化数（图 6），分别针对短提示和长提示。

结论：从图 6 可以看出，随着提示对的相似性增加，属性的平均变化数逐渐减少。在每个相似度区间内，名词的变化次数最多，其次是形容词和动词。相较于短提示，长提示在相同的相似性区间内发生了更多变化。这表明，即使提示文本中多个属性发生了变化，CLIP 嵌入依然未能准确区分这些变化较大的提示对。这反映了在处理非常长的提示时存在局限性，因为两个提示可能被误认为相似度较高。

会话内与会话间的查询相似度：我们首先将每个用户的提示进行分组，然后将该用户的提示按 1 小时的会话进行划分。接着，我们对比了同一用户同一会话内（intra）提示的相似性和不同会话间（inter）提示的相似性，结果见图 7a。此外，我们在图 7b 中将这些相似性得分与 100 个随机其他用户会话中的提示相似性进行了比较。

结论：图 7a 显示，许多用户在同一会话内表现出较高的提示相似性，表明用户在同一会话内通常会使用非常相似的提示，且频繁重复（图中表现为大量点聚集在相似度为 1 的区域）。同样，虽然用户在不同会话间的相似性也较高，但相较于会话内相似性，它要稍低一些，表明用户在不同会话中提出的查询更加多样化。在图 7b 中，我们观察到，同一用户在会话内的提示表现出较高的相似性。值得注意的是，不同会话间的提示，甚至来自不同用户的提示，也显示出显著的相似性。这表明，不同用户之间的提示相似性较高，暗示了利用群体力量进行近似缓存的潜力，有助于提升缓存效率。

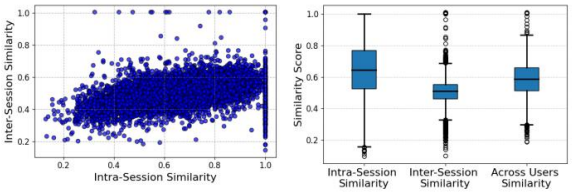


图 7: 查询提示相似性 (a) 在每个用户的会话间（inter-session）与会话内（intra-session）比较，(b) 各个用户之间的会话内与会话间相似性的范围。

4 NIRVANA 概述

4.1 近似缓存

对于新的查询提示 PQ, NIRVANA 使用近似学习来减少计算，通过检索在先验图像生成过程的第 K 次迭代后创建的中间状态，并在剩余的 N−K 步中直接重用和修复 PQ。

解析建模:设 L 表示使用近似缓存生成图像的总端到端延迟。其中，C 表示 N 个扩散模型步骤的 GPU 累计计算时间。K 的可能值的集合记为 K。矢量数据库(VDB)中的每个搜索操作都会产生一个表示为 ls 的延迟成本，从缓存中检索中间状态会引入一个表示为 lr 的延迟成本。我们使用 fc 来表示总体计算节省。

因此，对于有效利用近似缓存并在 K 处生成缓存的提示符，所经历的总延迟可以表示为:

$$l_s + C \cdot \frac{N-K}{N} + l_r \tag{5}$$

相比之下，NIRVANA 无法在缓存中找到匹配的提示将经历总延迟为 ls +C。
这种区别源于设计，它只在 VDB 中确认命中时才尝试从文件系统检索中间状态(导致延迟开销 lr)，以确保缓存中存在该状态。

让我们将近似缓存的 hit-rate@K 表示为 h(K)，它被定义为当使用 K 个扩散步骤的中间状态时，最多需要 N−K 个扩散步骤来生成一个忠实的修复图像，其中 N 是固定的。也就是说，存在(1−h(K))个部分的缓存，我们不能通过运行 N−K 步来修复。

现在在 K = 0(从头开始运行扩散模型)时，所有历史提示在理论上都是可用的，因为图像最多可以在 N−0 步中修复，从而导致 h(0) = 1.0。随着 K 的增加，h(K)减小，因为我们只能在第 K 步中使用更小的中间状态，通过最多 N−K 步的扩散来修复图像。当 K 值较低时，h(K)小于 1.0，但仍然可以相对较高。即，如果检索到的状态来自初始扩散步骤，则扩散模型可以有效地对检索到的状态进行重构，从而生成忠实的图像。

h(K)的减少受到提示符不同程度的影响。当 K 超过一定的阈值，记为 KT 时，检索到的状态不再适合进一步修复，如§2.2 所述，因此，h(K≥KT) = 0。

因此，对于给定 K, GPU 计算节省的有效分数可以表示为:

$$f_C = h(K) \cdot \frac{K}{N} \tag{6}$$

很明显，当 K 和 h(K)都足够高时，可以实现大量的节省。然而，挑战在于，随着 K 的增加，h(K)趋于减少，同时力求保持生成图像的质量 Q 1。图 9 展示了 DIFFUSIONDB 数据集[69]跨不同离散 K 值的趋势。

现在我们将 hopt(K)定义为在扩散步骤 K 中存储的缓存的分数，用于精确地在 N - K 步中修复图像。例如，当 N = 50,K = 5 时，我们得到 hopt(K)是可以用于通过运行扩散步骤精确 45 步来修复图像的缓存的百分比。因此,

$$h_{opt}(K) = h(K) - h(K'), \text{ where } \arg\min_{K'}(K' > K) \tag{7}$$

$$h(\min \mathcal{K}) = \sum_{K \in \mathcal{K}} h_{opt}(K) \tag{8}$$

本质上，$h_{opt}(K)$量化了 K 表示输入提示的最大潜在节省的概率。

h(minK)表示总体命中率，即缓存命中的 PQ 的百分比。我们的主要目标是最小化端到端延迟(L)，同时保持生成图像的质量(Q)。我们在这样的约束下操作:与从头生成图像相比，在选定的 K 值下使用缓存对图像进行修复必须确保指定级别的图像质量 Q。目标是找到满足这些目标和以下质量约束的最优 K 值。

因此，对于给定的传入提示 PQ 及其对应的缓存提示 Pc，目标(最小化 L):(根据 Eq. 5)

$$\min_K \mathcal{L} = \sum_{K \in \mathcal{K}} \left(l_s + h_{\text{opt}}(K) \cdot l_r + h_{\text{opt}}(K) \cdot \mathcal{C} \cdot \frac{N-K}{N} \right) \tag{9}$$

$$\mathbb{Q}(I_K^c | \mathcal{P}_{c_K}, \mathcal{P}_Q) > \alpha \cdot \mathbb{Q}(I_0 | \mathcal{P}_Q) \tag{10}$$

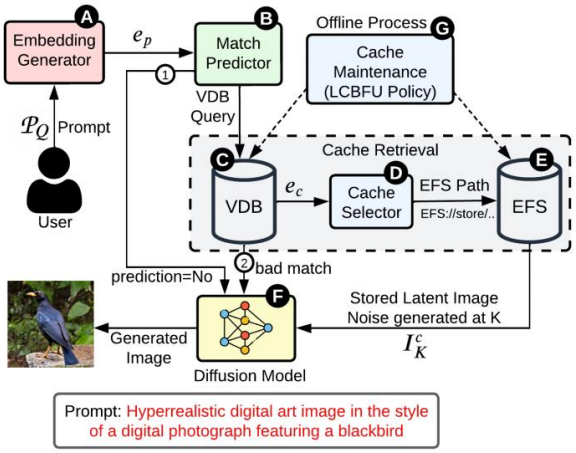


图 8:NIRVANA 概览

其中 i_k 表示在 K 处使用缓存 c，然后对 N−K 个扩散步骤进行修复所生成的图像。

α∈[0,1]表示对生成的图像质量的容忍阈值，使得 ick 并不比 I0 差多少。在我们的实现(§5.3.1)中，我们使用 CLIPScore 指标[40]来定义α = 0.9 的 Q。我们选择 CLIPScore 是因为它在评估图像质量方面的广泛使用。

在我们的用例中，$l_r, l_s << C$，目标降低为

$$\min_K \mathcal{L} = \sum_{K \in \mathcal{K}} \left(h_{\text{opt}}(K) \cdot C \cdot \frac{N-K}{N} \right) \quad (11)$$

它最大化 K 和 $h_{opt}(K)$以获得最小的延迟。

这个框架演示了在近似缓存环境下延迟和质量之间的关系。

值得注意的是，NIRVANA 浪费的开销可以捕获为$(1-h(K))\cdot l_s$，其中查询矢量数据库并导致缓存丢失。这意味着如果我们在 h(K)低的设置中操作，即使 K 很高，如果 ls 与 GPU 计算延迟 C 相当，NIRVANA 可能不会提供延迟优势，并且这种开销的减少可以是一个重要的设计方面，我们将在§5.2 中讨论。值得注意的是，由于高端 GPU 的不同成本，即使没有太多延迟减少，使用 NIRVANA 仍然可以显着降低成本，因为它大大降低了云上昂贵的 GPU 计算成本。在我们的设置中，我们观察到 ls 约为 100 毫秒，而 C 约为 10 秒。

4.2 系统组件

图 8 显示了 NIRVANA 的主要组件执行路径。首先，使用嵌入生成器 A 为当前输入提示符 PQ 生成嵌入向量 ep。然后，一个可选的匹配预测模块 B 预测向量数据库(VDB) C 中的嵌入是否有足够接近的匹配。

如果可能存在足够相似的缓存条目，那么 NIRVANA 将启动从缓存中检索中间状态的过程。首先，向 VDB 发送一个搜索查询，以查找最近的 ep 缓存嵌入，用 e_c 表示。VDB 使用高效的近似最近邻(ANN)[54]搜索来找到最接近的嵌入。对于 VDB 中缓存的每个历史提示，NIRVANA 在普通扩散过程中存储几个中间状态。对应于匹配提示符 e_c 的这些中间状态中，哪一个对于新的提示向量 ep 是最优的，由缓存选择器模块 D 使用启发式方法计算。然后，使用 VDB 查询的搜索结果所指向的存储位置指针和 cache-selector 模块计算出的特定中间状态编号(如§4.1 所述，即 K)，从存储器 E(即 NIRVANA 的弹性文件系统)中检索到这个特定的中间状态。中间状态 I c K 是在 ec 的去噪过程中捕获的 L × D 维的潜在表示，在第 K 步之后。最后，这个检索到的中间状态 I c K 与 ep 一起传递给扩散模型 F，以便对其进行修复，以便为剩余的去噪步骤生成图像。然而，在两种情况下，NIRVANA 会直接返回到普通扩散模型来从头开始生成图像，从而牺牲任何优化:(i) 当 match-predictor 模块预测 VDB 中不太可能有接近的条目时(箭头 1)，以及(ii)当 VDB 查询返回与 ep 非常不同的匹配 e_c 时(箭头 2)。

为了在固定大小的存储下维护缓存，防止 VDB 任意增长并继续存储过时的条目，缓存维护模块 G 在脱机模式下工作，并实现了新的 LCBFU 协议，以保持缓存和 VDB 条目的新鲜度。

4.3 NIRVANA 算法

算法 1 展示了 NIRVANA 的整体逻辑。首先，它使用匹配预测器来预测提示查询 PQ 是否存在紧密匹配。如果预测是肯定的，它会调用 VDB 来获得最近的提示符，并使用启发式方法找到 K(参见 5.2)。接下来，它在第 K 步从 EFS 中检索中间图像噪声，并将其传递给扩散模型，在 N−K 步中生成最终图像。如果没有预测或找到匹配，NIRVANA 会采取退步，从头开始生成图像。

5 NIRVANA 设计细节

现在我们展示 NIRVANA 的详细设计。

5.1 嵌入生成器

类似于传统的扩散模型，NIRVANA 首先从文本提示计算出一个 768 维的 CLIP 嵌入向量 e_p [59]。CLIP 有效地将视觉上相似的提示在嵌入空间中靠得更近，这在我们的案例中优化了缓存命中率的可能性。

5.2 匹配预测

使用e_p,NIRVANA 可以直接向向量数据库(VDB)发出搜索查询，以查找缓存中最接近的提示。如果存在缓存，则在图像生成中将显著节省 GPU 使用。然而，在缓存未命中的情况下，搜索 VDB 变成了一个延迟开销,而不会减少 GPU 计算。现在，多个因素会影响每次未命中时浪费的延迟开销的大小，包括：a) 是否在同一局域网内与 GPU 的距离，b) VDB 的特定架构及其内部索引机制，c) 分配给 VDB 的计算资源等。

为减少这种开销,NIRVANA 使用一个名为匹配预测器（match-predictor，见图 8 中的 B）的组件，它可以预测与 e_p 足够接近的嵌入是否可能出现在 VDB 中。如果预测结果表明不太可能，那么 NIRVANA 会完全跳过缓存检索流程，从而减少浪费的开销。此外，匹配预测器还减少了与搜索未命中对应的 VDB 负载，提高了可扩展性。

在内部，匹配预测器使用轻量级分类器进行预测,该分类器在与扩散模型在 GPU 上运行的同一节点的 CPU 上运行。这使得分类延迟比向 VDB 查询低几个数量级，从而使延迟开销变得微不足道。

回顾 § 4.1，从分析上讲，延迟开销为 $(1-h(k))\times l_s$。现在设 c_p 表示匹配预测器分类器的准确度。那么，具有活跃匹配预测器的 NIRVANA 的有效开销为：

$$1 - max(h(K), c_p) \cdot l_s, \text{ where } h(K), c_p \in [0, 1]$$

这意味着，当 h(K)=1.0h(K) = 1.0h(K)=1.0 时，即提示之间的相似度如此之高，以至于每个传入提示在缓存中都有合适的匹配，或者当 c_p=1.0 时，即分类器在预测中是完美的，系统将不会有任何浪费的延迟开销。需要注意的是，当 c_p<1.0 时，NIRVANA 在假阴性情况下会错失一些计算节省的机会，因为它将直接回退到传统扩散模型，并从头生成图像，而不是尝试检索中间状态。

对于这个分类器,NIRVANA 使用了一类支持向量机（One-Class Support Vector Machine, One-Class SVM）[20]，它为异常检测构建了决策函数。该分类器通过利用存储在 VDB 中的所有提示嵌入进行训练，并将它们分配为正标签 1。为了实现高精度和召回率，我们有效地使模型过拟合到现有的提示嵌入空间。此外，我们使用随机梯度下降（Stochastic Gradient Descent, SGD）来加速当 VDB 中的嵌入发生显著变化（即 > 5%）时对分类器的重新训练。NIRVANA 对实际生产提示达到了 c_p=0.95。

为什么不使用布隆过滤器？与我们的动机类似，布隆过滤器 [26] 被广泛用作在各种 Web 服务和数据库系统中检查缓存条目的低成本机制 [19]。布隆过滤器使用哈希算法来计算某个项是否可能存在于缓存中，其假阴性率为零，但存在一定的假阳性率。然而，布隆过滤器不适合 NIRVANA，因为我们并不搜索缓存条目的精确匹配，而是使用近似最近邻（ANN）算法在 VDB 中搜索最近邻。

Algorithm 1 GenerateImageCache($\mathcal{P}_Q$)

```
P_e ← Embed(P)
if MatchPredictor(P_e) is True then
    // Generate using cache
    (neigh, score) ← search_VDB(P_e)
    K ← heuristics_K(score)
    if K ≠ 0 then
        path_K ← neigh['payload']['noise'][K]
        c_noise ← retrieve_EFS(path_K)
        I ← model(P, c_noise, K)
    else
        // No suitable cache found, generate from scratch
        I ← model(P, null, 0)
    end if
else
    // Generate from scratch
    I ← model(P, null, 0)
end if
return I
```

5.3 缓存检索

缓存检索和缓存维护是 NIRVANA 的主要组件。在这一部分，我们详细阐述缓存检索阶段的设计,该阶段包含三个内部组件:向量数据库(VDB)、缓存选择器和存储系统。在 § 5.4 中，我们将讨论缓存维护器的设计。

向量数据库：NIRVANA 在向量数据库（VDB）中存储历史提示的嵌入，以便与传入提示的嵌入进行快速高效的相似性搜索。VDB 使用量化、图或树等索引方法来存储并对向量执行高维相似性搜索。对于查询嵌入向量，VDB 可以找到 m 个近似最近邻。在 NIRVANA 中，对于每个传入

搜索与 e_p ，VDB 已经填充了嵌入，其有效载荷指向在不同 K 处存储的相应中间状态的路径。NIRVANA 使用余弦相似度作为查找最近邻的度量。虽然 NIRVANA 可以与任何 VDB 一起工作，例如 Qdrant [21]、Milvus [18]、Weaviate [23] 和 Elasticsearch [13]，但在本文中，我们展示了与 Qdrant 的结果，因为基准测试 [3] 显示其在规模上具有较低的读取延迟，并且更新和删除延迟适中。

弹性文件系统作为存储：NIRVANA 将实际的中间噪声状态存储在 AWS 弹性文件系统 7(EFS)上。经过与 LustreFS[16]在读取延迟、吞吐量和存储成本方面的比较，我们选择了 EFS 作为我们的存储系统。包含中间状态的文件的大小取决于扩散模型的体系结构。对于 NIRVANA，默认扩散模型使用大小为 144 KB 的中间属性，并存储 K∈K ={5,10,15,20,25}的 5 个不同值。

5.3.1 缓存器

cache-selector 组件主要决定应该从 EFS 中检索哪个 K∈K for e_c，以在保持可接受的图像质量的同时，通过提示 PQ 来为剩下的 N−K 步重新调整中间状态。

我们怎么选择 K 呢?我们根据经验观察到，可以跳过的步骤数与 e_p 和 e_c 之间的相似性相关。基于这一观察，我们执行离线分析/表征(算法 2)，以找到合适的 K，使得在第 K 个扩散步骤生成的中间状态是基于两个提示符嵌入之间的相似性得分(Eq. 10)的最佳状态。该("CacheSelector-Profiling(.)")在每个 K 值处为一组提示生成图像，这些提示具有最近的缓存提示符。然后，它找到最小的相似度分数，使得所有生成的图像都高于质量阈值α。然后选择这个值作为最小相似度，使 K 的值起作用，并存储起来以供以后在运行时使用。如图 9 所示，我们将图像质量(越高越好)与多个 K 值之间的查询和缓存提示之间的各种相似度评分进行对比。

我们使用概要信息来确定在运行时生成图像的 K 的最佳值。利用从 VDB 中得到的 e_p 和 e_c 之间的相似度评分 s，我们选择一个 K，使得在 plot (x = K, similarity_score = s)处的质量评分≥α。我们选择α = 0.9。

简单地说，我们搜索算法 2 存储的逆映射。图 9 中的红色带表示阈值α。

图中的每条线对应不同的 K 值，在特定 K 值适用的特定相似度分数处与该红色带相交。因此，如果相似度得分高，我们可以跳过更多的扩散步骤，选择更高的 K 来获得可接受的图像质量。但是，如果相似度分数很低，我们只能跳过有限的迭代次数，因此可以得到较低的 K。图 10 显示了计算最优 K 的实际逻辑，给定两个提示之间的相似度分数 s，该 K 是使用前面讨论的分析技术为 DiffusionDB 数据集识别的。在此基础上，我们查询缓存存储，得到 I^c_K。

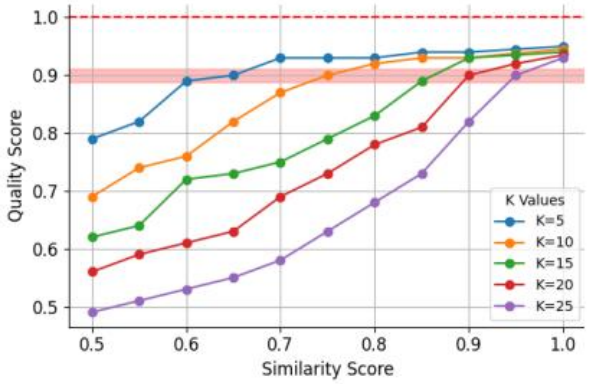
一旦 NIRVANA 从缓存中检索到适当的中间图像噪声，扩散模型去噪 N - K 步以生成最终图像。

质量与性能的权衡:NIRVANA 的设计目的是在不需要的情况下尽可能多地节省计算评估图像质量。然而，如果某些用例需要更激进的计算节省，并且愿意牺牲一些准确性，NIRVANA 可以轻松提供一个旋钮，用于在质量与计算节省之间进行权衡。缓存选择器的启发式算法可以倾向于选择较高的 K 值。这将提供更多的计算减少，同时让图像质量下降，因为 NIRVANA 将获得更少的步骤来根据新提示重新调整检索到的噪声。在极端情况下，当 K=N 时，NIRVANA 的行为将完全像一个纯图像检索系统 [4, 7]。

Algorithm 2 *CacheSelector*-Profiling($\mathcal{K}$, I^c_K, α)

1: **for** K in $\mathcal{K}$ **do**
2: $[I_K] \leftarrow \text{model}(\mathcal{P}_Q, I^c_K, K) \quad \forall \quad \mathcal{P}_Q$
3: $\text{min_sim} \leftarrow \min\{\text{sim } s \mid \forall I \in [I_K], \text{quality}(I) > \alpha\}$
4: $\text{sim_K_map}[K] \leftarrow \text{min_sim}$
5: **end for**
6: **return** sim_K_map

图 9：缓存与相似度得分在不同 KKK 值下的图像生成质量。不同 KKK 值下的缓存使用阈值。



```
def cache_selector(s):
  if s > 0.95: k = 25
  elif s > 0.9: k = 20
  elif s > 0.85: k =
     15
  elif s > 0.75: k =
     10
  elif s > 0.65: k = 5
  else: k = 0
  return k
```

图 10:根据相似度评分确定 K。

5.3 缓存维护:LCBFU 策略

在流行的生产文本到图像系统中，有大量的输入提示流。如果我们继续存储缓存，则与使用的总存储空间相关的成本会增加，并且 VDB 性能也会在超过一定的条目限制后下降。因此，尽管 NIRVANA 理论上可以支持无限缓存，但存储成本和不断增加的搜索延迟最终会使其失去吸引力。缓存维护组件在后台工作，以维护 EFS 和 VDB 中的缓存存储中的条目。

为了实现这一目标，NIRVANA 使用了一种新的缓存维护策略——最小计算效益和频繁使用(LCBFU)，该策略是为扩散模型中的近似缓存定制的。其基本思想是，并不是所有的中间状态都同样有利于节省总体计算和延迟。具有高 K 值的中间状态可以提供巨大的计算节省，但只能在 PQ 与缓存项之一具有高度相似性时使用。以低 K 存储的项可以用于各种提示，因为即使与 PQ 的相似性相对较低，也可以对它们进行修复，但节省的计算量很低。

使用 LCBFU, NIRVANA 可以在 EFS 中限制总共 1tb 的缓存，而不会降低任何性能或质量，这平均对应于存储在 VDB 中的 150 万个中间状态或噪声和 30 万个唯一提示嵌入。

传统缓存策略的缺点:由于独特的计算模型的逼近缓存，传统的缓存策略，如 LRU(最近最少使用)，LFU(最不经常使用)，FIFO(先进先出)在 NIRVANA 不是很有用，因为它们在这种情况下均匀地对待每个项目，只关注访问模式或到达顺序，同时驱逐一个项目。正如我们在§2.1 中所讨论的，命中率本身并不能决定 NIRVANA 作为具有大 K 值的项目的效率，即使不经常访问可以提供显着的总体计算减少。

LCBFU:我们设计的 LCBFU 既考虑了项目的访问频率，也考虑了命中时的潜在计算效益。它排除 LCBFU 得分最小的项目，每个项目 i 计算为:$f_i \times K_i$。其中 f_i 为第 i 项的访问频率，K_i 为该中间状态属于去噪过程的哪一步。值得注意的是，由于缓存选择器启发式基于 e_p 和 e_c 的相似性来确定 K，因此主动启发式(高 K 表示低相似性)将强制高 K 噪声的高保真度，因此低 K 噪声将被驱逐。例如，K_i= 25 的条目被访问 100 次，其 LCBFU 评分为 2500，而 K_j= 5 的条目被访问 200 次，其 LCBFU 评分为 1000。因此，项目 i 具有更高的 LCBFU 得分，因为它将在生成图像时提供更好的计算节省。LCBFU 的完整机制描述如下:

•插入:每次缓存缺失，我们直接将扩散去噪步骤 $K \in K$ 产生的所有中间状态插入到 cache-storage 中，并将相应的提示符嵌入到 VDB 中。因此，每个提示符都存储在缓存中。在达到目标存储限制之前，不执行任何插入操作。之后，每次插入之前都要进行一次驱逐。

• Eviction: 对于 Eviction，LCBFU 在一个 K-min 的堆中维护一个 LCBFU 评分的运行列表，并在插入 K 中间项之前从堆根中 e_v 出前 K 项。LCBFU 评分剔除对计算节省贡献最小的图像噪声。

使用此缓存清除策略，可能会出现这样的情况:某些 k 中的噪声被清除，而其他 k 中的噪声仍在缓存中。这会在存储的中间状态中产生漏洞。

处理漏洞:一旦驱逐策略创建了一个漏洞，就没有直接的方法来填补这个漏洞，因为如果不运行完整的扩散过程，就不可能实现中间状态的目标再生。然而，缓存选择器(§5.3.1)使用的启发式方法在确定与检索提示符一起使用的适当 K 时忽略了这些漏洞的存在。NIRVANA 通过选择 K 的最大值小于或等于最优 K 的中间状态来处理这种情况，这确保了 NIRVANA 继续生成高质量的图像，尽管在

遇到漏洞时几乎没有牺牲潜在的计算节省。然而，我们观察到这种情况只出现在 4-5%的提示中，导致难以察觉的性能下降。

NIRVANA 并没有采取积极的策略来清理这些漏洞。当一个提示符的所有中间状态与所有的 K 值对应时，LCBFU 将该提示符嵌入标记为脏的，并从 VDB 中删除它，同时从存储缓存中删除相应的元数据。

cache-maintainer 在 VDB 上批量执行插入和删除，同时匹配预测器中的分类器也使用 VDB 中的新条目进行重新训练。回想一下，正如§5.2 和§5.3 中提到的，VDB 更新和分类器更新都很快，对于 10k 记录分别只需要大约 7.5 和 0.04 秒。

6 实现

NIRVANA 是在 Python 中使用 PyTorch 实现的[56]。

批处理:在部署时，我们的扩散模型在批处理大小为 1 时占用了大约 80%的 GPU 内存，因此不支持并发批处理。

AITemplate (AIT): 我们利用 AITemplate (AIT)[11]，一个 python 框架，通过将其转换为 CUDA (NVIDIA GPU) / HIP (AMD GPU) c++代码来加速基于 pytorch 的扩散模型的推理服务。它在推理过程中提供高性能支持。

系统组件:我们的系统组件包括(i)使用 scikitlearn[57]中的 SGDOneClassSVM[20]的分类器，$\nu = 0.001$，其中ν控制训练误差和支持向量之间的权衡，(ii)托管在 AWS m5.4 4xlarge EC2 实例上的 Docker 容器中的 VDB, HNSW 索引为 256，用于快速嵌入搜索[54]，(iii)提供 web 和文件系统访问，对象存储和可扩展性的 AWS EFS 系统，(iv) MySQL 数据库记录对缓存项的访问。(v)文本到图像生成器，由更大版本的稳定扩散模型和 DDIM(去噪扩散隐式模型)采样器提供动力[66]。它在 $N = 50$ 次迭代中生成图像，使用大约 8 GB 的内存，批量大小为 1。

7 评估

我们首先评估 NIRVANA 在质量方面的总体有效性，以及为 AWS 云平台上的服务提供显著的计算和端到端延迟节省、吞吐量和成本节省。我们对 60 名参与者进行了用户研究，将图像生成质量与基线进行比较。我们展示了 LCBFU 对通用缓存管理策略的好处,以证明 NIRVANA 内部组件设计的有效性。我们也进行消融研究。关键要点是:• NIRVANA 生成高质量图像，同时减少 GPU 使用和端到端延迟高达 50%。

•NIRVANA 将扩散模型的所有三个成本、延迟和计算需求平均降低了约 20%。

•系统吞吐量提高 27%，NIRVANA 以最小的响应时间变化确保稳定的用户体验。

7.1 方法论

基础扩散模型:作为基础文本到图像模型，我们使用了一个更大的基于稳定扩散的模型(我们称之为 VANILLA)，与 2.3B 参数的稳定扩散 XL 模型相比，其参数数量约为 1.5 倍[58]。我们的模型在 96 像素的潜在空间内运行，在 A10g GPU 上执行 $N = 50$ 个去噪步骤，平均在 8.59 秒内生成图像。最终生成的图像大小为 768×768×3。

实验设置:我们在单个 NVIDIA A10g GPU (24gb GPU 内存)上运行 NIRVANA 的扩散模型和嵌入生成器，而其他组件在 32 核 AMD EPYC 7R32 2.8 GHz CPU 上运行，CPU 内存为 128gb。GPU 和 CPU 机器连接到一个包括 EFS 和 VDB 的子网。我们通过重复使用带有扩散模型嵌入的提示嵌入来优化 GPU 的使用，从而减少 GPU 开销。

数据集:我们对两个生产数据集进行评估。

•**DiffusionDB**[69]:总共 2M 张图像，1.5M 个唯一提示，总数据集大小≈1.6 tb。我们过滤了数据集并删除了 NSFW 图像。

•SYSTEM-X:来自生产设置 SYSTEM-X 的提示，跨度超过 8 周，包含超过 7M 张图像，6.2M 个唯一提示，总大小≈5 tb。

除非另有说明，否则我们在 DIFFUSIONDB 数据集上评估结果，因为 SYSTEM-X 数据是专有的。

基线:我们比较了两个版本的 NIRVANA:(1)有匹配预测器(称为 NIRVANA)和(2)没有匹配预测器(称

为 NIRVANA-w/oMP)针对以下基线:

•GPT-CACHE:基于 BERT 嵌入相似度检索最接近提示的图像。否则，从头开始生成图像[7]

•PINECONE:根据 CLIP 文本嵌入相似度检索最接近提示的图像。

•CRS (Clip Retrieval System，剪辑检索系统):剪辑检索系统[5]是另一种图像检索方法，它在检索给定输入提示最接近的图像时，使用由前面提示生成的最终图像的嵌入。

•SMALLMODEL:一个更小的扩散模型[60]，有 860M 个参数，与 VANILLA 相比只消耗 33%的计算/延迟，在 A10g 上平均在 3.05 秒内通过 50 个扩散步骤生成图像。

工作负载生成器和缓存预加载:工作负载生成器按照它们到达的顺序分配提示，使用数据集的到达时间戳字段来创建提示流。每个提示查询在前一个查询完成后立即发送给 NIRVANA。只有在使用了最初的 10k 提示来预加载 VDB 和 EFS 及其各自的缓存以启动系统之后，这个查询流才会开始。

评估指标:我们根据涵盖质量和效率方面的各种指标来评估 NIRVANA。

1 .质量指标

•FID 评分(fr<s:1>起始距离):计算两个图像数据集之间的差异，并与人类视觉质量感知相关[41]。

•CLIP 评分:评估生成图像与其文本提示之间的对齐[40]

•PickScore:用于预测生成图像的用户偏好的度量[46]

2.效率指标:为了评估 NIRVANA 在提供系统效率方面的功效，我们报告了平均 5 次运行，节省了%的 GPU 使用时间，减少了%的图像生成的端到端延迟，在集群中每秒生成的图像数量增加了%的吞吐量，并且还平摊了每次图像生成的成本。我们还报告了 NIRVANA 的总体命中率=h(minK) (Eq. 8)。

7.2 整体表现质量

在本节中,我们评估 NIRVANA 如何在图像生成质量方面表现得更好。

定量生成性能:表 1 总结了 NIRVANA 在定量图像质量指标方面的改进。当与基于检索的基线(§7.1) 进行比较时，NIRVANA 和 NIRVANA-w/oMP 在所有三个指标中都显著提高了性能[67]。基于检索的基线直接检索由最相似的前一个查询提示生成的图像。因此，这些方法无法捕获输入提示和检索提示之间的差异，因为相似性度量无法捕获这些差异(§3)。因此，所有基于检索的基线都会对生成的图像质量产生重大影响，这在面向生产用户的文本到图像用例中是至关重要的。与 SMALLMODEL 相比，NIRVANA 和 NIRVANA-w/oMP 的性能都要优越得多，这表明大型模型即使使用近似缓存也能保持质量。

VANILLA 的指标显示了生成的图像质量，没有任何近似。与 CLIP 和 Pick 分数相比，NIRVANA 在这两个数据集上的表现非常接近 VANILLA。这表明近似缓存不会损害整体性能。

我们根据相同提示使用 VANILLA 生成的图像来测量基线的 FID(越低越好)。我们还计算了香草的 FID，以表明在不改变基本模型的情况下生成的图像的内在可变性。

为了计算 VANILLA 的 FID，我们生成了 4 组带有 4 种不同种子的图像，并计算了它们之间的 FID

Dataset	Models	Quality		
		FID ↓	*CLIP Score* ↑	*PickScore* ↑
DiffusionDB	GPT-CACHE	7.98	25.84	19.04
	PINECONE	10.92	24.83	18.92
	CRS	8.43	24.05	18.84
	SMALLMODEL	11.14	25.64	18.65
	NIRVANA − *w/o*MP	4.94	28.65	20.35
	NIRVANA	**4.68**	**28.81**	**20.41**
	VANILLA	6.12-6.92	30.28	20.86
SYSTEM-X	GPT-CACHE	8.15	26.32	19.11
	PINECONE	10.12	24.43	18.83
	CRS	8.38	23.81	18.78
	SMALLMODEL	11.35	25.91	18.92
	NIRVANA − *w/o*MP	4.48	28.94	20.31
	NIRVANA	**4.15**	**29.12**	**20.38**
	VANILLA	5.42-6.12	30.4	20.71

表 1:我们将 NIRVANA 与几个基线 GPTCACHE，PINECONE，CRS 进行比较，这些基线是纯基于检索的技术，较小的模型和基本的香草扩散模型。CLIP 分数是根据文本到图像的分数计算的。所有其他的是图像之间的比较，使用香草图像作为地面真相。分类器的准确率为 0.96，召回率为 0.95。

产生 C_4^2 组图像。从这两个数据集可以看出，NIRVANA 和 NIRVANA-w/oMP 的 FID 值比 VANILLA 模型中不同世代之间的内部差异要低得多。这意味着 NIRVANA 生成的图像将与 VANILLA 模型无法区分-这是设计目标。

NIRVANA的性能比NIRVANA-w/oMP稍好一些，因为 match-predictor 会在预测缓存丢失时从头生成图像。然而，与 NIRVANA-w/oMP 相比，这对 NIRVANA 的效率有很小的影响(见§7.3)。

获取质量的用户研究:我们进行了一项有 60 名参与者的用户研究，以证明定性分析。我们评估了从DIFFUSIONDB中随机选择的大约1000个提示，其中每个用户都有 15 个随机的<提示，生成的图像>对，如果他们认为生成的图像正确地代表了给定的提示，则要求他们投票赞成或反对。在这 15 对提示图像中，5 对图像是由性能最好的基于图像检索基线的 GPT-CACHE 生成的，接下来的 5 对图像是由 NIRVANA 生成的，5 对图像是使用 VANILLA 生成的。这三种类型的问题被打乱，并以随机顺序呈现给每个用户，并且在用户会话中不会重复提示。用户还可以选择披露拒绝的原因。

在图 11 中，我们显示了在所有用户中聚合的三个模型中的每个模型的Yes和No响应之间的比率。可以看出 GPT-CACHE 获得的赞成票最少，而 NIRVANA 仅略低于上限 VANILLA。当分析 No 原因(约 12%的负面反应)时，我们发现这些原因中有 80%是由 GPT-CACHE 生成的图像。
这些见解强调了仅仅依靠基于检索的图像生成解决方案的挑战，因此我们在进一步的评估中不使用这些基线。

这加强了 NIRVANA 在保持图像质量方面的优势，这对商业部署至关重要。

7.3 NIRVANA 的系统效率

我们从图像生成延迟、命中率、计算节省和运行 NIRVANA 的成本等方面比较了 NIRVANA。

延迟:图 2 中的延迟指的是测试的 200 万个提示查询的平均端到端延迟。我们在一些基线的响应延迟的中位数上绘制了第 n 个百分位数。图像生成系统应该表现出低延迟，以增强用户体验。然而，像 GPT-CACHE 这样的纯检索技术提供了低延迟，但产生的图像质量较低，并且端到端延迟波动较大，因为它需要对某些与缓存提示不同的提示重新运行扩散模型，从而影响用户体验[24]，如表 2 所示。与基线相比，NIRVANA 减少了这种延迟差异，并且与 VANILLA 相比，总体延迟也减少了，提供了更稳定、更快的用户体验。最小化响应时间可变性的关键在于计算时间在不同提示之间的平滑转换，这要归功于 NIRVANA 在不同 K 值下检索缓存的能力。

这一观察结果强调了 NIRVANA 不仅将总延迟减少了 19.8%，而且还将不同提示之间的响应时间差异最小化，最终提供一致且稳定的用户体验[27]。

命中率和计算节省:图 15 突出显示了 NIRVANA 在 K (x 轴)上的命中率和计算节省。

在所有 k 的平均值上，NIRVANA 达到了 88%的命中率，并且值得注意的是，如蓝色和黑色虚线所示，节省了 21%的计算时间。累积命中率曲线说明了命中率随不同 K 值和图 h(K)的变化情况。此外，条形图显示了在每个 K 下可以实现的潜在节省以及在特定 K 值下实现的实际节省。例如，当 K = 25 时，可能节省 50%，而实际命中率为 8%，因此实际节省 4%。

吞吐量:我们将吞吐量量化为系统每分钟处理的提示数量。为了评估这一点，我们重播了来自 DiffusionDB 数据集的提示流。考虑到两种缓存设置，我们评估了 NIRVANA 与 VANILLA 的相对吞吐量。第一种设置使用缓存大小为 150 万项的 LCBFU，而第二种设置涉及一个不断增加的缓存配置，其中不会发生缓存驱逐，从而有效地提供理论上无限的缓存大小。图 12 表示了我们的发现，其中 x 轴对应于查询流，我们绘制了系统的相对吞吐量。

值得注意的是，我们的结果表明，在这两种设置下，NIRVANA 的吞吐量都比 VANILLA 高~1.28 倍。

图像生成成本:为了提供 NIRVANA 和 VANILLA的详细成本细目，我们根据AWS定价[1]考虑了特定的 AWS 组件及其相关费用。我们使用来自美国东部地区的 g5.24xlarge GPU (96 GB GPU)实例，每小时花费 8.144 美元。

此成本用于估算 VANILLA 和 NIRVANA 产生的 gpu 相关费用。VANILLA 只使用 GPU 资源生成每个图像。然而，NIRVANA 还依赖于 GPU 之外的其他组件，即 VDB[6]和 EFS。对于 VDB，成本估计为每小时 0.12 美元，其中包括在提示嵌入上执行的存储和搜索操作。此外，NIRVANA 还会产生与 EFS 相关的成本。在美国东部地区，使用标准存储类型和 20%的频繁访问超过弹性吞吐量成本相当于每小时 0.09 美元。为了确定 NIRVANA 的总体平摊成本，我们通过将 GPU 成本除以吞吐量，然后加上与 VDB 和 EFS 相关的成本来计算。值得注意的是，该成本分析是在两个系统的相同设置下进行的。尽管包含了这些额外的费用，图 13 突出显示，与 VANILLA 相比，NIRVANA 的成本显著降低了 19%。这种显著的成本效率强调了 NIRVANA 在实际部署中的实际优势。

7.4 LCBFU 性能

我们针对常见的缓存技术(如 FIFO、LRU 和 LFU)对 LCBFU 的有效性进行了实验。为了确保公平的评估，我们在所有使用冷缓存设置的实验中保持相同的工作负载生成器设置。FIFO 从缓存中移除最早添加的噪声。LRU 和 LFU 根据缓存项的访问频率和最近次数来清除缓存项。表 3 报告了不同缓存大小的各种缓存清除策略的命中率(公式 8)和%计算节省(与 VANILLA 相比 GPU 时间节省的%)。LCBFU 的命中率与 LRU 和 LFU 相当，优于 FIFO。值得注意的是，如表(b)中所强调的，与所有其他策略相比，拟议的 LCBFU 提供了大量的计算节省，因为它旨在将 K 与图像访问频率结合起来进行驱逐。LCBFU 的设计并不是为了获得最佳命中率。

7.5 NIRVANA 中端到端延迟的分解

在图 17 中，我们展示了 VANILLA 扩散模型的端到端延迟，以及 NIRVANA 的不同组件对其端到端延迟的贡献。我们可以看到，来自 EFS 的噪声检索和 VDB 搜索是开销的主要贡献者。

7.6 长提示与短提示之间的图像质量

正如我们在§3 中讨论过的，提示查询可以长也可以短。我们进行消融，看看系统是如何与它们一起工作的。表 4 所示的消融结果表明，与非常长的提示相比，较短的提示更有效地执行 NIRVANA。这种性能上的差异可以直观地归因于所采用的检索技术。较长的提示往往会挑战嵌入(在我们的例子中是 CLIP)充分捕捉上下文的能力。尽管基于 CLIPText 嵌入的相似性很高(如§3 所述)，但如果从缓存中检索到的提示符明显偏离查询，则可能导致生成不连贯的图像

Prompt	$FID \downarrow$	$CLIPScore \uparrow$
Short	7.96	28.48
Long	11.48	28.96

表 4：生成短 v/s 长的提示。少于 15 个单词被认为是简短的。

7.7 敏感性分析

我们现在提出一些关于 NIRVANA 设计选择的敏感性分析。

匹配预测器的有效性:我们对使用匹配预测器的有效性进行了评估。在 NIRVANA 中，我们测

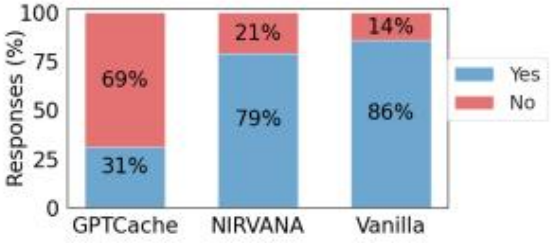


图 11：用户调查响应

	Latency(s)	$90^{th}/median$	$95^{th}/median$	$99^{th}/median$
GPTCache	2.8	29.52	29.64	29.75
NIRVANA	6.9	1.20	1.21	1.21
VANILLA	8.6	1.01	1.02	1.02

表 2：方法的平均延迟和响应延迟中位数的第 n 个百分位数

量了平均图像生成延迟。图 16 显示了 match-predictor 对降低尾部延迟值的显著贡献。此外，相当一部分查询的开销可以忽略不计，几乎为零。这种行为归因于匹配预测器能够迅速预测某

个特定提示是否存在于缓存中。它减少了对 I/ o 相关活动的需求，从而降低了延迟。

匹配预测器的设置:sgdoneclasssm 匹配预测器可以通过使用各种阈值产生二进制预测(0 或 1)。这些阈值影响从匹配预测器获得的精度(P)和召回率(R)值。为了确定最佳设置，我们进行了消融研究，测量了不同 P 和 R 配置下的开销延迟。图 18 中的结果图使我们选择了 P = 96 和 R = 95

嵌入类型的设置:我们对使用不同类型的查询和/或缓存嵌入的 NIRVANA 进行了比较。一些嵌入应用于查询和缓存提示，而其他嵌入仅应用于缓存的提示，并且为查询提示维护 CLIP 嵌入。这是由“提示/缓存”列表示的。表 5 总结的结果表明，通过 FID、CLIP Score 和 PickScore 等指标评估的图像生成质量在不同的嵌入类型中保持一致。然而，CLIP 文本嵌入的命中率和计算节省明显优于其他两种嵌入类型。因此，我们选择 CLIP 文本嵌入作为我们的系统设计在 NIRVANA 中，我们

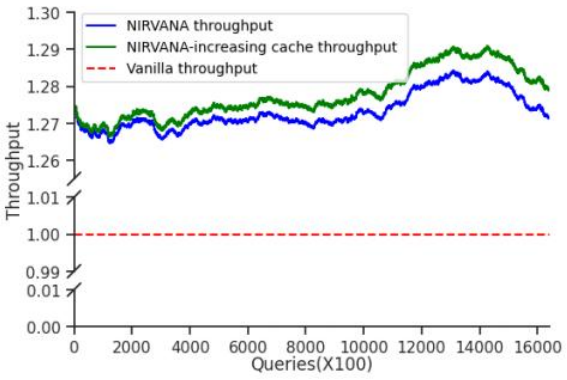


图 12:针对查询流的模型吞吐量比较

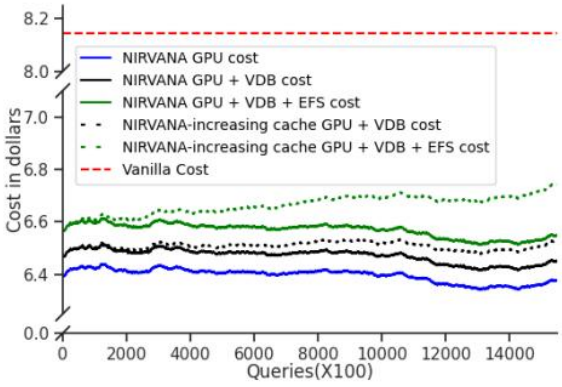


图 13:针对查询流的模型组件的成本比较

Cache Size(GB)	#noises in cache	*FIFO*	*LRU*	*LFU*	LCBFU
1*GB*	1500	0.58	0.65	0.64	0.65
10*GB*	15000	0.68	0.77	0.78	0.77
100*GB*	150000	0.74	0.85	0.83	0.82
1000*GB*	1500000	0.83	0.95	0.94	0.93

(a) Hit rate

Cache Size(GB)	#noises in cache	*FIFO*	*LRU*	*LFU*	LCBFU
1*GB*	1500	0.11	0.12	0.12	0.12
10*GB*	15000	0.13	0.14	0.14	0.15
100*GB*	150000	0.14	0.16	0.16	**0.18**
1000*GB*	1500000	0.17	0.20	0.19	**0.23**

(b) % Compute savings

表 3:不同驱逐技术的性能。使用 LCBFU 回收可以显著节省计算量

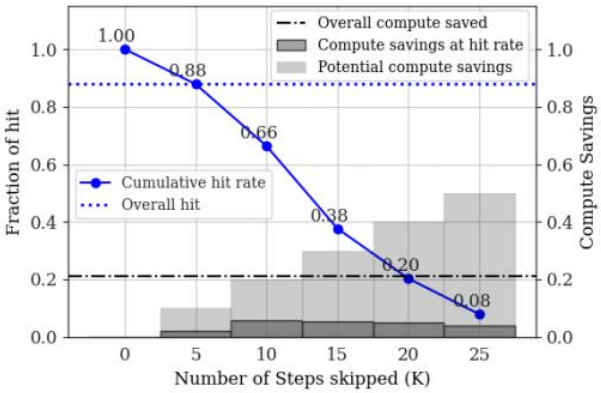


图 15:跨越 K 的命中率和计算

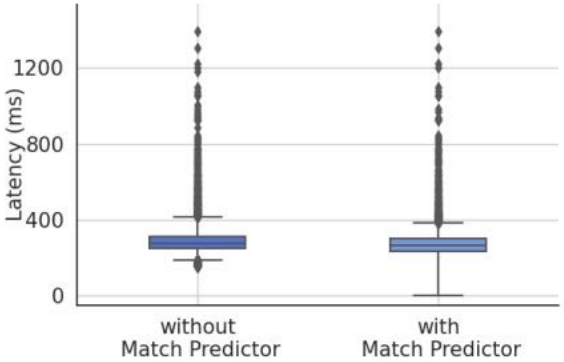


图 16:使用和不使用 match-predictor 的延迟分布。

测量了平均图像生成延迟。图 16 显示了 match-predictor 对降低尾部延迟值的显著贡献。此外，相当一部分查询的开销可以忽略不计，几乎为零。这种行为归因于匹配预测器能够迅速预测某

个特定提示是否存在于缓存中。它减少了对 I/ o 相关活动的需求，从而降低了延迟。

匹配预测器的设置:sgdoneclasssm 匹配预测器可以通过使用各种阈值产生二进制预测(0 或 1)。这些阈值影响从匹配预测器获得的精度(P)和召回率(R)值。为了确定最佳设置，我们进行了消融研究，测量了不同 P 和 R 配置下的开销延迟。图 18 中的结果图使我们选择了 P = 96 和 R = 95 嵌入类型的设置:我们对使用不同类型的查询和/或缓存嵌入的 NIRVANA 进行了比较。一些嵌入应用于查询和缓存提示，而其他嵌入仅应用于缓存的提示，并且为查询提示维护 CLIP 嵌入。

这是由“提示/缓存”列表示的。表 5 总结的结果表明，通过 FID、CLIP Score 和 PickScore 等指标评估的图像生成质量在不同的嵌入类型中保持一致。然而，CLIP 文本嵌入的命中率和计算节省明显优于其他两种嵌入类型。因此，我们选择 CLIP 文本嵌入作为我们的系统设计。

Embedding	Prompt/Cache	FID↓	CLIPScore↑	HitRate↑	ComputeSavings↑
BERT	Query & Cache	4.53	28.94	0.77	0.16
CLIPImage	Only Cache	4.85	28.49	0.84	0.18
CLIP	Query & Cache	4.94	28.65	**0.93**	**0.23**

表 5:不同嵌入的生成质量。

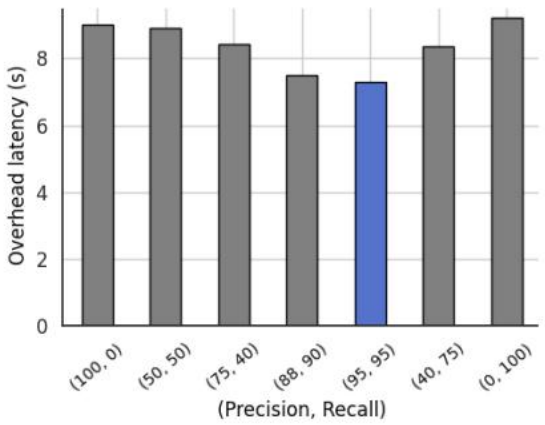


图 18:跨越 K 的命中率和计算

7.8 不同缓存策略的质量

我们进行了评估，以比较图像质量指标，包括 FID, CLIPScore 和 PickScore，同时使用不同的缓存清除技术。表 6 所示的结果表明，在所有缓存技术中，图像生成质量保持一致。因此，选择驱逐机制应该优先考虑提高计算节省和命中率

Policy	*FID* ↓	*CLIPScore* ↑	*PickScore* ↑
FIFO	5.12	28.25	20.31
LRU	4.82	28.54	20.38
LFU	4.98	28.61	20.42
LCBFU	4.94	28.65	20.41

表 6:使用 1500 GB 缓存的不同清除技术的生成质量。

8 相关工作

机器学习优化：各种优化技术，如模型蒸馏[55,63]、剪枝[35]、量化[52]和其他[43,50,73]存在于大型扩散模型中，但往往也会降低质量。DeepSpeed[32]是一个用于分布式推理的优化库，并实现了 13 种相同的技术。这些可以使基本模型更快，但与 NIRVANA 是正交的，因为它们并没有从根本上改变 NIRVANA 利用计算减少的迭代去噪过程的本质。

云中的模型服务:过去的研究[30,39,51,71,74]探索了有效的 ML 模型，用于减少推理延迟。Clipper[30]实现了层缓存、集成方法和散列缓解等优化。Cocktail[39]设计了一个具有成本效益的基于集成的模型服务框架，以及用于资源管理的主动自动伸缩，以提供高通达性和低延迟。其他工作[28,29,74]优化了负荷变化时的管道成本预测。然而，这些都是对 NIRVANA 的补充，可以很容易地集成。Tabi[68]在大型语言模型中使用了多个异构模型。

Text-to-image 模型扩散模型是文本到图像模型的主要类别之一，由 Dall-E[12]、Imagen[62]和 Stable Diffusion[60]推广开 Midjourney[17]、Deci[15]和 Adobe[9]等其他几家企业也建立了自己的扩散模型。算法优化包括设计更快的采样步骤[42,47,66,70,77]和并行采样[65,77]，从而以计算换取速度。然而，我们的工作是正交的，底层的扩散模型可以从上述任何工作中选择。

缓存过去已经探索了 DNN 推理中的缓存

[30,38,48,49]，包括缓存中间层输出，以避免在不同输入上再次运行每层[30,49]。Kumar 等人[49]提出了近似缓存这一术语，但我们的语义是正交的，因为我们缓存的是中间图像噪声，而不是模型层的输出。在这些工作中实现的缓存策略(LRU、静态缓存)在我们的案例中都不起作用。其他缓存技术[25,72]对于我们的目的来说是非常重要的。基于检索的作品[4,5,7]使用缓存来检索图像，以获得最相似的提示，但质量较差(参见§7)。

9 讨论

图像的多样性。由于 NIRVANA 重用缓存中以前的中间状态，如果系统遇到的大多数提示非常相似，那么随着时间的推移，系统生成的图像的多样性可能会减少。这可以通过两种方式解决:第一，在检索后主动改变去噪过程的种子。我们观察到，当 K < 35 时，这可以增加多样性。如果提示变得过于相似，那么 NIRVANA 将尝试使用更高的 K 来更积极地节省计算，这样种子更改将变得无效。在这种情况下，NIRVANA 可以设计成让 ab ε分数的提示遵循香草扩散过程，以保持图像的多样性。

提示符特征的改变。如果提示的特征由于某些外部因素而突然改变，NIRVANA 将在缓存中找到更少的相似项，并将自动移动到更低的 K，甚至对于更多不同提示，match-predictor 将启动并将提示重定向到香草扩散过程。虽然这将减少计算节省，但根据设计 NIRVANA 不会让图像质量在提示特性突然变化的情况下下降。

机器学习优化。一些技术被用来减少模型的计算占用和延迟，如使用低精度[52]、蒸馏[55,63]、剪枝[35]、批处理推理优化[22]。NIRVANA 是对这些技术的补充，因为它可以在这些优化模型之上使用，也可以使用近似缓存来减少冗余计算。然而，一个新的生成模型家族出现了，它不需要这么多的迭代步骤，那么 NIRVANA 的适用性就会受到限制。但是，由于到目前为止，具有 50 个或更多去噪步骤的扩散模型可以产生最佳和高质量的图像，因此 NIRVANA 为减少计算量提供了一个有吸引力的建议。

10 结论

在本文中，我们介绍了 NIRVANA 的设计和实现，它使用一种称为近似教学的新技术，通过缓存和重用在处理先前文本提示时创建的中间状态，使用扩散模型显着降低了文本到图像生成过程中的计算成本和延迟。我们还提出了一种新的缓存管理技术来管理这些中间状态，以在固定存储限制下优化性能。

参考文献

[1] Aws product and service pricing | amazon web services. https://aws.amazon.com/pricing/?aws-products-pricing.sort-by=item.additionalFields.productNameLowercase&aws-products-pricing.sort-order=asc&awsf.Free%20Tier%20Type=*all&awsf.tech-category=*all. (Accessed on 09/20/2023).

[2] Complete guide to samplers in stable diffusion - félix sanz. https://www.felixsanz.dev/articles/complete-guide-to-sampler s-in-stable-diffusion. (Accessed on 09/21/2023).

[3] Hugging face model repository. https://huggingface.co/models.

[4] Making stable diffusion faster with intelligent caching | pinecone. https://www.pinecone.io/learn/faster-stable-diffusion/.(Accessed on 09/21/2023).

[5] rom1504/clip-retrieval: Easily compute clip embeddings and build a clip retrieval system with them. https://github.com/rom1504/c lip-retrieval. (Accessed on 09/21/2023).

[6] Vector search database | qdrant cloud. https://cloud.qdrant.io/ calculator. (Accessed on 09/20/2023).

[7] zilliztech/gptcache: Semantic cache for llms. fully integrated with langchain and llama_index. https://github.com/zilliztec h/GPTCache. (Accessed on 09/21/2023).

[8] Adobe express with ai-powered firefly integration now commercially available. https://news.adobe.com/news/news-details/2023/Adobe-Express-With-AI-Powered-Firefly-Integration-Now-Commercially-Available/default.aspx, 2023.

[9] Adobe firefly. https://www.adobe.com/sensei/generative-ai/firefly.html.2023.

[10] Adobe unleashes new era of creativity for all with the commercial release of generative ai. https://news.adobe.com/news/news-details/2023/Adobe-Unleashes-New-Era-of-Creativity-for

-All-With-the-Commercial-Release-of-Generative-AI/default.aspx, 2023.

[11] Aitemplate. https://github.com/facebookincubator/AITemplate, 2023.

[12] Dall-e 2. https://openai.com/dall-e-2. 2023.

[13] Elasticsearch. https://www.elastic.co/, 2023.

[14] Intel labs introduces ai diffusion model, generates 360-degree images from text prompts. https://www.businesswire.com/news/home/20230621842353/en/Intel-Labs-Introduces-AI-Diffusion-Model-Generates-360-Degree-Images-from-Text-Prompts, 2023.

[15] Introducing decidiffusion 1.0: : 3x the speed of stable diffusion with the same quality. https://deci.ai/blog/decidiffusion-1-0-3x-faster-than-stable-diffusion-same-quality/. 2023.

[16] Lustrefs. https://www.lustre.org/, 2023.

[17] Midjourney. https://www.midjourney.com/home/. 2023.

[18] Milvus - vector database. https://milvus.io/, 2023.

[19] Myrocks and bloom filters. https://mariadb.com/kb/en/myrocks-and-bloom-filters/, 2023.

[20] Oneclass svm. https://scikit-learn.org/stable/modules/generated/sklearn.svm.OneClassSVM.html, 2023.

[21] Qdrant - vector database. https://qdrant.tech/, 2023.

[22] Stable diffusion batch prediction with ray data. https://docs.ray.io/en/latest/data/examples/stablediffusion_batch_prediction.html, 2023.

[23] Weaviate - vector database. https://weaviate.io/, 2023.

[24] Xiao Bai, Ioannis Arapakis, B Barla Cambazoglu, and Ana Freire. Understanding and leveraging the impact of response latency on user behaviour in web search. *ACM Transactions on Information Systems (TOIS)*, 36(2):1–42, 2017.

[25] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. LHD : Improving cache hit rate by maximizing hit density. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages389–403, 2018.

[26] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[27] Peter M Broadwell. *Response time as a performability metric for online services*. Computer Science Division, University of California, 2004.

[28] Lequn Chen, Weixin Deng, Anirudh Canumalla, Yu Xin, Matthai Phili- pose, and Arvind Krishnamurthy. Symphony: Optimized model serv- ing using centralized orchestration. *arXiv preprint arXiv:2308.07470*, 2023.

[29] Daniel Crankshaw, Gur-Eyal Sela, Corey Zumar, Xiangxi Mo, Joseph E. Gonzalez, Ion Stoica, and Alexey Tumanov. Inferline: Ml prediction pipeline provisioning and management for tight latency objectives, 2020.

[30] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. Clipper: A Low-Latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 613–627, 2017.

[31] Florinel-Alin Croitoru, Vlad Hondru, Radu Tudor Ionescu, and Mubarak Shah. Diffusion models in vision: A survey. *IEEE Transac- tions on Pattern Analysis and Machine Intelligence*, 2023.

[32] deepspeed.ai. Deepspeed. https://www.deepspeed.ai/, 2023.

[33] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*,34:8780–8794, 2021.

[34] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weis- senborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[35] Gongfan Fang, Xinyin Ma, and Xinchao Wang. Structural pruning for diffusion models. *arXiv preprint arXiv:2305.10924*, 2023.

[36] Paul A Gagniuc. *Markov chains: from theory to implementation and experimentation*. John Wiley & Sons, 2017.

[37] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Gen-erative adversarial nets. In *Advances in Neural Information ProcessingSystems*, 2014.

[38] Edouard Grave, Armand Joulin, and Nicolas Usunier. Improving neural language models with a continuous cache. In *International Conference on Learning Representations*, 2017.

[39] Jashwant Raj Gunasekaran, Cyan Subhra Mishra, Prashanth Thi- nakaran, Bikash Sharma, Mahmut Taylan Kandemir, and Chita R Das. Cocktail: A multidimensional optimization for model serving in cloud. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 1041–1057, 2022.

[40] Jack Hessel, Ari Holtzman, Maxwell Forbes, Ronan Le Bras, and Yejin Choi. Clipscore: A reference-free evaluation metric for image caption- ing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7514–7528, 2021.

[41] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale up- date rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30, 2017.

[42] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion proba- bilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[43] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2021.

[44] Jina.ai. Benchmark vector search databases with one million data. https://jina.ai/news/benchmark-vector-search-databases-with-one-million-data/, 2022.

[45] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[46] Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. *arXiv preprint arXiv:2305.01569*, 2023.

[47] Zhifeng Kong and Wei Ping. On fast sampling of diffusion proba- bilistic models. In *ICML Workshop on Invertible Neural Networks, Normalizing Flows, and Explicit Likelihood Models*, 2021.

[48] Roland Kuhn. Speech recognition and the frequency of recently used words: A modified Markov model for natural language. In *Coling Budapest 1988 Volume 1: International Conference on Computational Linguistics*, 1988.

[49] Adarsh Kumar, Arjun Balasubramanian, Shivaram Venkataraman, and Aditya Akella. Accelerating deep learning inference via freezing. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*, 2019.

[50] Fan Lai, Yinwei Dai, Harsha V Madhyastha, and Mosharaf Chowdhury. ModelKeeper : Accelerating DNN training via automated training warmup. In *NSDI*, 2023.

[51] Yunseong Lee, Alberto Scolari, Byung-Gon Chun, Marco Domenico Santambrogio, Markus Weimer, and Matteo Interlandi. PRETZEL : Opening the black box of machine learning prediction serving systems. In *OSDI*, 2018.

[52] Xiuyu Li, Yijiang Liu, Long Lian, Huanrui Yang, Zhen Dong, Daniel Kang, Shanghang Zhang, and Kurt Keutzer. Q-diffusion: Quantizingdiffusion models. *arXiv*, 2023.

[53] Luping Liu, Yi Ren, Zhijie Lin, and Zhou Zhao. Pseudo numer- ical methods for diffusion models on manifolds. *arXiv preprint arXiv:2202.09778*, 2022.

[54] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approxi- mate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelli- gence*, 42(4):824–836, 2018.

[55] Chenlin Meng, Robin Rombach, Ruiqi Gao, Diederik Kingma, Stefano Ermon, Jonathan Ho, and Tim Salimans. On distillation of guided diffusion models. In *CVPR*, 2023.

[56] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Brad- bury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[57] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion,
O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vander-plas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duch- esnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[58] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. Sdxl: im-proving latent diffusion models for high-resolution image synthesis. *arXiv preprint arXiv:2307.01952*, 2023.

[59] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.

[60] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffu-sion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[61] O Ronneberger, P Fischer, and T Brox. Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015 Conference Proceed-ings*, 2022.

[62] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily L Denton, Kamyar Ghasemipour, Raphael Gontijo Lopes, Burcu Karagol Ayan, Tim Salimans, et al. Photorealistic text-to-image diffu- sion models with deep language understanding. *Advances in Neural Information Processing Systems*, 35:36479–36494, 2022.

[63] Tim Salimans and Jonathan Ho. Progressive distillation for fast sam- pling of diffusion models. In *International Conference on Learning Representations*, 2021.

[64] Erich Schubert, Jörg Sander, Martin Ester, Hans Peter Kriegel, andXiaowei Xu. Dbscan revisited, revisited: why and how you should (still) use dbscan. *ACM Transactions on Database Systems (TODS)*, 42(3):1–21, 2017.

[65] Andy Shih, Suneel Belkhale, Stefano Ermon, Dorsa Sadigh, and Nima Anari. Parallel sampling of diffusion models, 2023.

[66] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representa- tions*, 2020.

[67] Jianyi Wang, Kelvin CK Chan, and Chen Change Loy. Exploring clipfor assessing the look and feel of images. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 2555–2563,2023.

[68] Yiding Wang, Kai Chen, Haisheng Tan, and Kun Guo. Tabi: An efficient multi-level inference system for large language models. In *Proceedings of the Eighteenth European Conference on Computer Systems*, pages233–248, 2023.

[69] Zijie J Wang, Evan Montoya, David Munechika, Haoyang Yang, Ben- jamin Hoover, and Duen Horng Chau. Diffusiondb: A large-scale prompt gallery dataset for text-to-image generative models. *arXiv preprint arXiv:2210.14896*, 2022.

[70] Zike Wu, Pan Zhou, Kenji Kawaguchi, and Hanwang Zhang. Fast diffusion model, 2023.

[71] Neeraja J Yadwadkar, Francisco Romero, Qian Li, and Christos Kozyrakis. A case for managed and model-less inference serving. In *Proceedings of the Workshop on Hot Topics in Operating Systems*,pages 184–191, 2019.

[72] Juncheng Yang, Yao Yue, and Rashmi Vinayak. Segcache: a memory- efficient and scalable in-memory key-value cache for small objects. In *18th USENIX Symposium on Networked Systems Design and Imple- mentation (NSDI 21)*, pages 503–518, 2021.

[73] Miao Yin, Yang Sui, Siyu Liao, and Bo Yuan. Towards efficient ten- sor decomposition-based dnn model compression with optimization framework. In *CVPR*, 2021.

[74] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan. MArk : Exploiting cloud services for Cost-Effective , SLO-Aware machine learning inference serving. In *2019 USENIX Annual Technical Confer- ence (USENIX ATC 19)*, pages 1049–1062, 2019.

[75] Chenshuang Zhang, Chaoning Zhang, Mengchun Zhang, and In So Kweon. Text-to-image diffusion model in generative ai: A survey.*arXiv preprint arXiv:2303.07909*, 2023.

[76] Yuxin Zhang, Weiming Dong, Fan Tang, Nisha Huang, Haibin Huang, Chongyang Ma, Tong-Yee Lee, Oliver Deussen, and Changsheng Xu. Prospect: Expanded conditioning for the personalization of attribute- aware image generation. *arXiv preprint arXiv:2305.16225*, 2023.

[77] Hongkai Zheng, Weili Nie, Arash Vahdat, Kamyar Azizzadenesheli, and Anima Anandkumar. Fast sampling of diffusion models via operator learning. In *International Conference on Machine Learning*, pages 42390–42402. PMLR, 2023.